

Rishihood University

Computer Networks – Course Project

Private Network Service Platform

A Two-Phase, Team-Based Local Networking Project

Phase 1: Build & Observe | Phase 2: Harden & Recover | Final: Explain & Defend

“The application stays simple – the network is the project.”

Team ID:	team1
Private domain:	app.team1.test, api.team1.test
Team members:	Member 1 Name (Roll No.) Member 2 Name (Roll No.) Member 3 Name (Roll No.) Member 4 Name (Roll No.)
Faculty:	Faculty Name
Date:	October 3, 2026

Master Document: Architecture, Configuration, Evidence Index, Failure Analysis, Phase 2 Report and
Viva Preparation

Abstract

This master document records the design, implementation, observation and hardening of a fully local private service environment built on up to four macOS laptops connected to one private LAN. A client resolves the private name `app.team1.test` through the team's own DNS server (dnsmasq), opens an HTTPS connection to an nginx edge node that terminates TLS and load-balances round-robin across two REST backends, and receives a response whose path can be proven layer by layer with `dig`, `curl`, browser developer tools and Wireshark. Phase 1 delivers the working platform with packet evidence (DNS $\rightarrow$ TCP $\rightarrow$ TLS $\rightarrow$ HTTP), caching behaviour and five deliberate failure experiments. Phase 2 adds a backup DNS resolver, DNS TTL experiments, backend service isolation with firewall rules, passive-health-check failover, DNS-based edge migration and a systematic troubleshooting methodology. The document doubles as the Architecture Document, Configuration Bundle index, Evidence Folder index, Phase 2 Final Report and viva-preparation guide.

Contents

1	Introduction and Project Scope	6
1.1	Purpose	6
1.2	Learning Outcomes	6
1.3	Project Constraints and Ground Rules	6
1.4	Team Responsibility Matrix	7
1.5	Two-Phase Structure at a Glance	8
2	Architecture Document	9
2.1	Network Topology	9
2.2	Machine Roles and Cloud Equivalents	10
2.3	IP and Service Inventory	10
2.4	DNS Records	10
2.5	Request Flow	11
2.6	Protocol-to-Layer Mapping (OSI vs. TCP/IP)	11
2.7	Moving Data Through the Core	12
3	Phase 1 – Build and Observe	13
3.1	Course-Topic Mapping	13
3.2	Task A – Establish the Private LAN	14
3.3	Task B – Configure a Private DNS Server (Mac 1)	15
3.4	Task C – Build Two Simple Backend Services (Mac 3 and Mac 4)	16
3.5	Task D – Edge Reverse Proxy and Load Balancer (Mac 2)	17
3.6	Task E – Add HTTPS / TLS	18
3.7	Task F – Demonstrate HTTP Caching Behaviour	20
3.8	Task G – Capture the Complete Protocol Flow	22
3.9	Explanation-Only Topics	24
4	Phase 1 – Required Failure Demonstrations	25
4.1	Failure 1 – Wrong DNS Server on a Client	25
4.2	Failure 2 – DNS Record Points to a Wrong IP	25

4.3	Failure 3 – One Backend Stopped	25
4.4	Failure 4 – Both Backends Stopped	26
4.5	Failure 5 – Wrong Destination Port on the Client	26
4.6	Failure Summary Table	26
5	Phase 2 – Harden, Recover and Troubleshoot	28
5.1	Extension A – Backup DNS Resolver	28
5.2	Extension B – DNS TTL and Controlled Record Change	29
5.3	Extension C – Service Isolation (Backend Firewall Rules)	30
5.4	Extension D – High-Availability Failover Behaviour	30
5.5	Extension E – Controlled Edge Migration (DNS-Based Cutover)	31
5.6	Extension F – Faculty-Injected Troubleshooting Challenge	32
6	Final Demonstration (11-Step Sequence)	34
6.1	Pre-Demo Readiness Checklist	35
7	Deliverables, Evidence Index and Marks	36
7.1	Deliverables	36
7.2	Suggested Repository Layout	36
7.3	Evidence Index (30-second rule)	37
7.4	Marks Breakdown and Self-Assessment	38
8	Viva Preparation	39
8.1	Phase 1 Concepts	39
8.2	Phase 2 and Whole-System Concepts	40
9	Phase 2 Final Report	42
9.1	What Changed from Phase 1	42
9.2	Resilience Test Results	42
9.3	Troubleshooting Findings	43
9.4	Learning Summary (one paragraph per extension)	43
9.5	Conclusion	43
A	Configuration Files	44
A.1	dnsmasq – Primary (Mac 1)	44
A.2	nginx – Phase 1 and Phase 2 (Mac 2)	44
A.3	Backend Application (Mac 3 and Mac 4)	46
A.4	pf Rules (Extension C)	46
B	Command Cheat Sheet	47

C Wireshark Filter Reference	48
D Glossary	49
E Execution Plan for Completion	50

List of Figures

2.1	Local topology: four roles on one private LAN segment.	9
2.2	Request flow: Client → DNS (Mac 1) → HTTPS (Mac 2, nginx) → Backend A (Mac 3) or Backend B (Mac 4).	11
3.1	Task A: ipconfig/ifconfig output on each Mac	14
3.2	Task A: ping results between all machine pairs	15
3.3	Task B: dig app.team1.test resolving to Mac 2's IP via Mac 1	16
3.4	Task C: Backend A and Backend B responses with X-Backend header	17
3.5	Task D: six consecutive requests alternating between Backend A and B	18
3.6	Task E: curl -v showing successful certificate verification	20
3.7	Task E: browser padlock with certificate details	20
3.8	Task F: Cache-Control and ETag headers via curl -I	21
3.9	Task F: conditional request returning 304 Not Modified	21
3.10	Task F: browser DevTools showing cached response	21
3.11	Task G: DNS query and response (UDP/53)	23
3.12	Task G: TCP three-way handshake (SYN, SYN-ACK, ACK)	23
3.13	Task G: TLS handshake packets	23
3.14	Task G: encrypted application data	23
4.1	Failure demos: screenshots for all five scenarios	27
5.1	Extension A: resolution via backup DNS while primary is stopped	29
5.2	Extension B: old answer cached, then new answer after TTL expiry, then after flush	29
5.3	Extension C: Mac 2 reaches backend, client Mac blocked, name still works	30
5.4	Extension D: Backend A stopped, all requests served by B; then A restored	31
5.5	Extension E: old edge answering, then standby after TTL expiry	32

List of Tables

1.2	Machine ownership (edit names; combine roles if the team has fewer than four Macs).	7
1.3	Phase comparison.	8
2.1	Roles mapped to cloud concepts.	10
2.2	Address inventory (fill from Task A commands).	10
2.3	Service map.	10
2.4	Private zone records. Clients never see backend IPs.	10
2.5	Protocol-to-layer mapping. TLS is commonly placed between application and transport; the exact OSI layer (5/6) is debated – state this in the viva.	11
3.2	Ping reachability matrix (tick when verified).	14
4.1	Failure scenarios and isolation by layer.	26
5.2	Fault catalogue to practise before the evaluation.	33
7.2	Evidence index.	37
7.3	Marks breakdown from the course brief.	38
9.1	Differences between Phase 1 and Phase 2.	42
9.2	Phase 2 test log (complete during rehearsal).	42
E.1	Suggested working order and time budget.	50

Chapter 1

Introduction and Project Scope

1.1 Purpose

The project gives hands-on experience with how a real network request travels from a client to a server and back. Instead of using cloud services or pre-configured servers, the team builds a private service environment from scratch on its own laptops. By the end, a client machine on the team network:

- types a private domain name (`app.team1.test`) into a browser or `curl`;
- resolves the name through the *team's own DNS server*;
- establishes a secure HTTPS connection through the team's *reverse proxy*;
- receives a response from *one of two backend application servers*;
- allows every step of the journey to be observed with packet capture tools.

1.2 Learning Outcomes

After both phases every team member must be able to:

1. **Translate theory into configuration** – configure and run DNS, TCP, TLS and HTTP rather than only read about them.
2. **Explain where each protocol fits** in a single client request (DNS, TCP, TLS, HTTP, caching, load balancing).
3. **Prove behaviour with tools** – `curl`, `dig`, Wireshark and browser DevTools. “It works” is not evidence.
4. **Diagnose failures by isolating layers** – DNS, TCP, TLS or application.
5. **Understand the whole system** – every member can explain any component, not only the one they configured.

1.3 Project Constraints and Ground Rules

Constraint	Rule and how this project complies
Team size	1 to 4 students (maximum 4). Teams of 2–3 may combine machine roles.
Hardware	Up to 4 macOS laptops; no dedicated server or cloud VM.
Network	All machines on the same private Wi-Fi/LAN (lab Wi-Fi, home router or controlled hotspot).
Cloud hosting	Not used. Everything is demonstrable locally; cloud concepts are discussed only as analogies.
Admin access	The DNS machine and the edge machine need administrator rights to install software and change network/DNS settings.
Application complexity	Backends are minimal REST/JSON APIs; the network is evaluated, not the app.
Port restrictions	If ports 80/443 cannot be bound, use 8080/8443. No marks are deducted.
Private domain	Reserved <code>.test</code> namespace (<code>team1.test</code>). Never use <code>.local</code> (conflicts with macOS mDNS).
Suggested tools	Homebrew, dnsmasq, nginx, OpenSSL/mkcert, Python or Node.js, curl, dig/nslookup, Wireshark.

1.4 Team Responsibility Matrix

Although each member owns one machine role, **every member must understand every component** (individual viva carries 20 of 100 marks).

Machine	Owner	Primary role	Secondary duties
Mac 1	Member 1 Name (Roll No.)	Private DNS + test client	Backup-DNS config sync, W client captures
Mac 2	Member 2 Name (Roll No.)	Edge: nginx reverse proxy, LB, TLS	CA creation, nginx configs, pf-ing
Mac 3	Member 3 Name (Roll No.)	Backend A (port 3001)	Standby edge (Ext. E), back (Ext. A)
Mac 4	Member 4 Name (Roll No.)	Backend B (port 3002) + test client	Evidence folder, report writing

Table 1.2: Machine ownership (edit names; combine roles if the team has fewer than four Macs).

1.5 Two-Phase Structure at a Glance

	Phase 1: Build & Observe	Phase 2: Harden & Recover
Focus	Build core infrastructure; prove it with tools.	Add resilience, isolation, DNS failover and controlled-failure diagnosis.
End result	Local service reachable by private domain with full DNS→TCP→TLS→HTTP packet evidence.	Robust network that survives selected failures and can be diagnosed when faults are injected.
Gate	Client resolves <code>app.team1.test</code> , connects over HTTPS and receives responses from both backends via the load balancer.	Backup DNS, TTL behaviour, service isolation, HA failover and at least one troubleshooting challenge demonstrated.

Table 1.3: Phase comparison.

Chapter 2

Architecture Document

2.1 Network Topology

All four Macs attach to one private Wi-Fi/LAN segment. The roles are a *network topology*, not a software architecture.

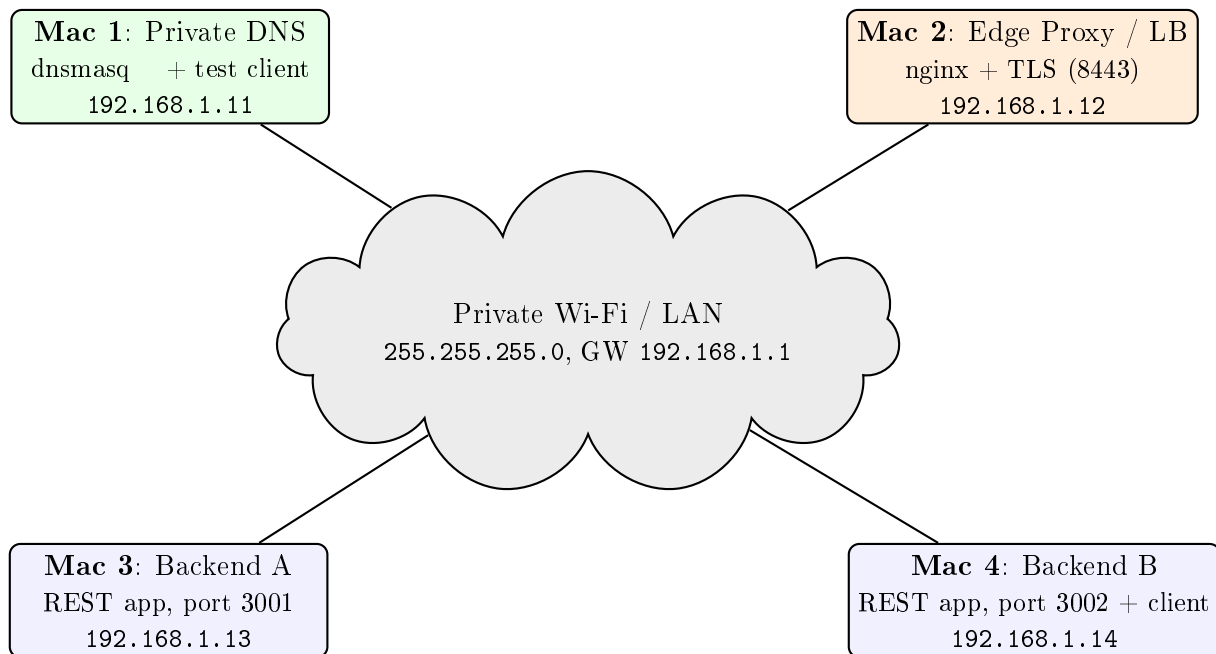


Figure 2.1: Local topology: four roles on one private LAN segment.

2.2 Machine Roles and Cloud Equivalents

Mac	Primary role	Services	Cloud equivalent
1	Private DNS server + test client	dnsmasq, dig, curl/browser	Managed DNS (Amazon Route 53)
2	Edge reverse proxy + load balancer	nginx, TLS certificate	Cloud load balancer / CDN edge (AWS ALB, GCP LB)
3	Backend server A	REST app, port 3001	Application server instance A
4	Backend server B + test client	REST app, port 3002, curl/browser	Application server instance B

Table 2.1: Roles mapped to cloud concepts.

2.3 IP and Service Inventory

Mac	IPv4	Mask	Gateway	Iface	MAC address
1 (DNS)	192.168.1.11	255.255.255.0	192.168.1.1	en0	[TODO: aa:bb:cc:...]
2 (Edge)	192.168.1.12	255.255.255.0	192.168.1.1	en0	[TODO: aa:bb:cc:...]
3 (Backend A)	192.168.1.13	255.255.255.0	192.168.1.1	en0	[TODO: aa:bb:cc:...]
4 (Backend B)	192.168.1.14	255.255.255.0	192.168.1.1	en0	[TODO: aa:bb:cc:...]

Table 2.2: Address inventory (fill from Task A commands).

Service	Host	Port/Proto	Bind address	Purpose
dnsmasq (primary)	Mac 1	53/UDP+TCP	192.168.1.11	Authoritative private zone
dnsmasq (backup)	Mac 3 or 4	53/UDP+TCP	its LAN IP	Phase 2 backup resolver
nginx HTTPS	Mac 2	8443/TCP	0.0.0.0	TLS termination, LB
nginx HTTP redirect	Mac 2	8080/TCP	0.0.0.0	Redirect to HTTPS (optional)
Backend A	Mac 3	3001/TCP	0.0.0.0	REST app (X-Backend: A)
Backend B	Mac 4	3002/TCP	0.0.0.0	REST app (X-Backend: B)

Table 2.3: Service map.

2.4 DNS Records

Name	Type	Value
app.team1.test	A	192.168.1.12 (Mac 2, edge)
api.team1.test	A	192.168.1.12 (Mac 2, edge)

Table 2.4: Private zone records. Clients never see backend IPs.

2.5 Request Flow

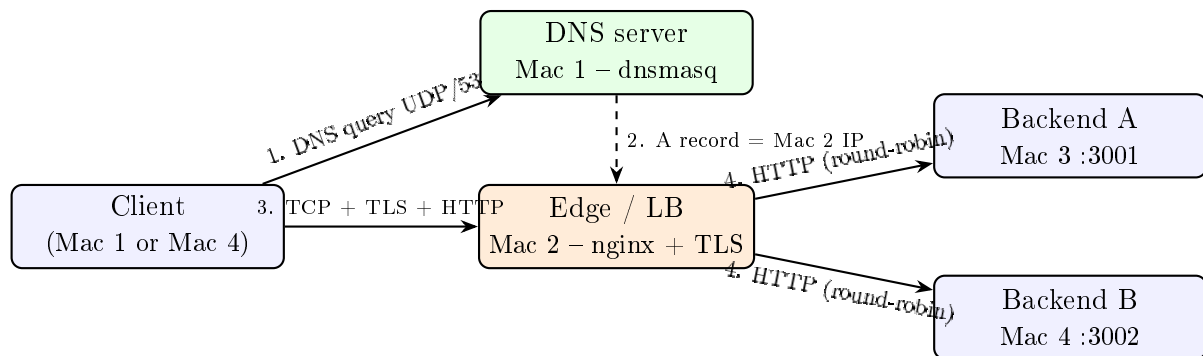


Figure 2.2: Request flow: Client → DNS (Mac 1) → HTTPS (Mac 2, nginx) → Backend A (Mac 3) or Backend B (Mac 4).

1. The client's stub resolver asks Mac 1 for `app.team1.test` (UDP/53).
2. Mac 1 answers with Mac 2's IP address. DNS only *finds* the address; it does not connect.
3. The client opens a TCP connection to Mac 2 port 8443 (three-way handshake).
4. A TLS handshake follows: the edge presents a certificate for `app.team1.test` signed by the team CA.
5. The HTTP request is sent encrypted inside TLS. nginx terminates TLS.
6. nginx forwards the request (plain HTTP/1.1) to Backend A or Backend B in round-robin order.
7. The backend answers with JSON and an `X-Backend` header; nginx relays it to the client.

2.6 Protocol-to-Layer Mapping (OSI vs. TCP/IP)

OSI layer	TCP/IP layer	Protocols / artefacts in this project
7 Application	Application	DNS, HTTP/1.1, HTTP/2, REST/JSON, Cache-Control, ETag
6 Presentation	Application	TLS encryption and certificate encoding (X.509)
5 Session	Application	TLS session establishment/resumption
4 Transport	Transport	TCP (ports 8443, 3001, 3002, DNS-over-TCP); UDP (port 53)
3 Network	Internet	IPv4 addressing, ICMP (ping), default gateway
2 Data link	Link	Ethernet / Wi-Fi frames, MAC addresses, ARP
1 Physical	Link	Wi-Fi radio / LAN cable

Table 2.5: Protocol-to-layer mapping. TLS is commonly placed between application and transport; the exact OSI layer (5/6) is debated – state this in the viva.

2.7 Moving Data Through the Core

A request leaves the application, is wrapped by TLS (encryption), then TCP (segments with ports, sequence numbers), then IP (source/destination addresses), then Ethernet/Wi-Fi (MAC addresses of the next hop). Because all hosts share one subnet, the frame is addressed directly to the destination's MAC (resolved by ARP) and no router forwards it beyond the LAN. The default gateway is only used for destinations outside the subnet. Each hop on the return path performs the reverse (decapsulation). This is captured in Wireshark in Task G.

Chapter 3

Phase 1 – Build and Observe

Success condition

Phase 1 is complete when a client resolves `app.team1.test`, connects over HTTPS, and receives responses from **both** backends through the load balancer.

3.1 Course-Topic Mapping

Course topic	What this project shows
Moving data through the core	Trace a request leaving the client, crossing the LAN, reaching the edge, returning (Wireshark).
OSI vs. TCP/IP	Map DNS, HTTP, TLS, TCP/UDP, IP, Ethernet to layers.
Devices, topologies, cloud concepts	Local topology diagram; each role related to its cloud equivalent.
HTTP/1.1, HTTP/2, REST	REST API; HTTP/1.1 demonstrated; HTTP/2 via nginx; HTTP/3 explanation only.
HTTPS and TLS	TLS terminated at nginx; handshake and certificate validation captured and explained.
DNS and Route 53 concepts	Private DNS name; at least two clients resolve through the team DNS server.
Transport layer; ports; TCP/UDP	Service ports, socket pairs, TCP three-way handshake captured.
Reliable data transfer; flow control	Sequence/ACK numbers and window size in Wireshark.
Email protocols; CDNs and caching	Email explanation-only; caching via Cache-Control and 304 responses.
Cloud load balancing	nginx as local LB, related to AWS ALB / GCP Load Balancer.

3.2 Task A – Establish the Private LAN

Objective

Place all Macs on one private network and verify reachability. Nothing else works until this is verified.

Procedure

1. Connect all Macs to the same private Wi-Fi/LAN (lab Wi-Fi, home router or hotspot).
2. On every Mac record IPv4 address, mask, gateway, interface name and MAC address:

```
ipconfig getifaddr en0          # private IPv4 address
ifconfig en0 | grep -E "inet | ether" # IPv4, netmask (hex) and MAC address
netstat -nr | grep default      # default gateway
networksetup -listallhardwareports # confirm the interface name (en0 / en1)
```

3. Ping every pair of machines (12 directed pairs for 4 Macs):

```
ping -c 4 <OTHER_MAC_IP>
```

4. Draw the topology diagram (Figure 2.1).

Note

`ifconfig` prints the netmask in hexadecimal (e.g. `0xffffffff00` = `255.255.255.0` = `/24`). Disable VPNs. If the router has “client isolation” / “AP isolation” enabled (common on public Wi-Fi), Macs cannot ping each other – use a personal hotspot or home router instead.

From \ To	Mac 1	Mac 2	Mac 3	Mac 4
Mac 1	–	☐	☐	☐
Mac 2	☐	–	☐	☐
Mac 3	☐	☐	–	☐
Mac 4	☐	☐	☐	–

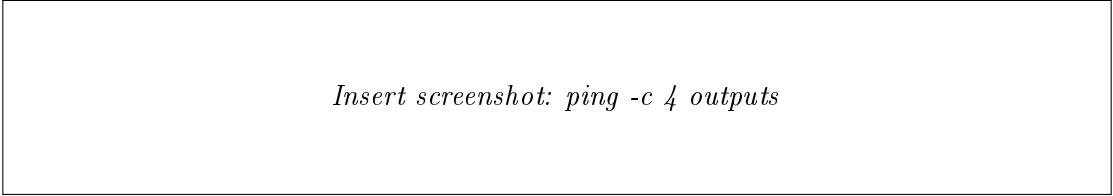
Table 3.2: Ping reachability matrix (tick when verified).

Evidence to collect

IP table (Table 2.2); ping screenshots for all pairs; topology diagram.

Insert screenshot: terminal output with IP, mask, gateway, MAC

Figure 3.1: Task A: `ipconfig/ifconfig` output on each Mac



Insert screenshot: ping -c 4 outputs

Figure 3.2: Task A: ping results between all machine pairs

Be ready to explain (viva)

What is a subnet mask? Why do hosts on the same subnet reach each other without a router? What does ARP do? What is the difference between a private and a public IPv4 address?

3.3 Task B – Configure a Private DNS Server (Mac 1)

Theory

DNS is a distributed directory that maps names to IP addresses. A *stub resolver* on the client sends a UDP query to port 53 of the configured DNS server and receives an answer (A record) with a TTL. Our private zone `team1.test` lives on Mac 1 and acts like a tiny managed DNS service such as Route 53 private hosted zones. The `.test` TLD is reserved (RFC 6761) for testing and will never be delegated on the public Internet.

Procedure

1. Install and configure dnsmasq on Mac 1:

```
brew install dnsmasq
# config file path: $(brew --prefix)/etc/dnsmasq.conf
```

2. Append the records (full file in Appendix A.1):

```
listen-address=<MAC1_IP>,127.0.0.1
bind-interfaces
address=/app.team1.test/<MAC2_IP>
address=/api.team1.test/<MAC2_IP>
log-queries
log-facility=/tmp/dnsmasq.log
```

3. Validate and start (port 53 needs root):

```
dnsmasq --test
sudo brew services start dnsmasq      # restart after edits
sudo lsof -nP -iUDP:53                # confirm dnsmasq is listening
```

4. Point at least two other Macs to Mac 1 as their DNS resolver:

```
# GUI: System Settings -> Network -> Wi-Fi -> Details -> DNS -> add <MAC1_IP>
# CLI equivalent:
networksetup -setdnsservers Wi-Fi <MAC1_IP>
networksetup -getdnsservers Wi-Fi
scutil --dns | head -30                # verify the resolver actually in use
```

5. Verify from a client:

```
dig app.team1.test
dig +short app.team1.test @<MAC1_IP>
nslookup app.team1.test
dscacheutil -q host -a name app.team1.test    # uses the macOS system resolver
```

6. Access the application only by name, never by typing the IP.

Note

dig talks directly to the DNS server and *bypasses* the macOS resolver cache. **curl**, **ping**, browsers and **dscacheutil** go through the system resolver (mDNSResponder). Keep this in mind in Extension B. If a Mac still cannot resolve the name, check **scutil -dns** for a stale resolver, turn off VPN/iCloud Private Relay, and make sure macOS Firewall is not blocking dnsmasq.

Evidence to collect

dnsmasq config; **dig**/**nslookup** output from two different clients; **scutil -dns** or Network-settings screenshot.

Insert screenshot: dig output showing SERVER = Mac 1 IP and the ANSWER section

Figure 3.3: Task B: **dig app.team1.test** resolving to Mac 2's IP via Mac 1

Be ready to explain (viva)

Difference between *DNS resolution* (finding the IP) and the *TCP/HTTPS connection* that follows. What does an A record contain? What is a TTL? Why is DNS usually UDP/53 and when does it use TCP? What is the Route 53 equivalent of this dnsmasq setup?

3.4 Task C – Build Two Simple Backend Services (Mac 3 and Mac 4)

Requirements

Endpoint	Expected response
GET /	Basic JSON/page confirming the service is running.
GET /api/status	JSON like {"backend": "A", "status": "ok"}.
GET /api/catalog	Static, cacheable JSON with Cache-Control and ETag (Task F).
Every response	Header X-Backend: A (or B).

Backends bind to 0.0.0.0 (not 127.0.0.1) on fixed ports: A = 3001, B = 3002. The full source is in Appendix A.3.

Run and verify

```
python3 -m venv venv && source venv/bin/activate && pip install flask
# Mac 3
BACKEND_ID=A BACKEND_PORT=3001 python backend.py
# Mac 4
BACKEND_ID=B BACKEND_PORT=3002 python backend.py

# From ANOTHER Mac (proves it is LAN-reachable)
curl -i http://<MAC3_IP>:3001/api/status
curl -i http://<MAC4_IP>:3002/api/status
```

Evidence to collect

Source code (GitHub repo or zip); `curl -i` output from another machine showing the X-Backend header and JSON body; `lsof -nP -iTCP:3001 -sTCP:LISTEN` showing a bind to `*:3001`.

Insert screenshot: curl -i output from a different Mac

Figure 3.4: Task C: Backend A and Backend B responses with X-Backend header

Be ready to explain (viva)

Why must the backend bind to `0.0.0.0`? What is the difference between listening on `127.0.0.1` and a LAN address? What is a socket (IP + port + protocol)? Why do the backends use different ports?

3.5 Task D – Edge Reverse Proxy and Load Balancer (Mac 2)

Theory

A *reverse proxy* sits in front of servers and accepts client connections on their behalf; a *load balancer* spreads those requests across several backends. With **round-robin**, requests are handed to upstreams in turn (A, B, A, B, ...). **least_conn** sends each request to the upstream with the fewest active connections. Cloud equivalents are AWS ALB/NLB or the GCP Load Balancer. Clients never need backend IPs because DNS points only at the edge; backends can be added, removed or moved without any client change.

Procedure

1. `brew install nginx`; the configuration lives in `$(brew -prefix)/etc/nginx/nginx.conf`.
2. Define the **upstream** block with both backends and a **server** block (full file in Appendix A.2).
3. Validate and reload:

```
sudo nginx -t
sudo nginx -s reload          # or: sudo nginx (first start)
```

4. Test round-robin from a client:

```
for i in $(seq 1 6); do
  curl -s -D - -o /dev/null https://app.team1.test:8443/api/status | grep -i x-backend
done
# expected: A, B, A, B, A, B
```

Evidence to collect

nginx config; repeated-request output showing alternating X-Backend: A/B; `nginx -t` success.

Insert screenshot: terminal loop output

Figure 3.5: Task D: six consecutive requests alternating between Backend A and B

Be ready to explain (viva)

Why does the client never need backend IPs? Round-robin vs. least_conn vs. ip_hash. Which layer does this load balancer operate at (L7 HTTP vs. L4 TCP)? How does it compare with a cloud ALB? What is the single point of failure here?

3.6 Task E – Add HTTPS / TLS

Theory

TLS provides *confidentiality* (encryption), *integrity* (MAC/AEAD) and *authentication* (certificate). The client trusts a server certificate only if it chains to a CA in its trust store and the certificate's Subject Alternative Name (SAN) matches the requested hostname. Our team CA is private, so its root certificate must be installed on every client. The edge *terminates* TLS: client-edge traffic is encrypted; edge-backend traffic is plain HTTP on the LAN.

Handshake summary. ClientHello (versions, ciphers, SNI, key share) → ServerHello (chosen version/cipher, key share) → Certificate (+ proof of key possession) → Key exchange (ephemeral Diffie-Hellman) → Finished (both sides). After this, application data is encrypted with session keys derived from the key exchange.

Procedure – Option 1: mkcert (needs faculty approval)

```
brew install mkcert nss
mkcert -install                                # creates local CA, trusts it on THIS Mac
mkdir -p ~/certs && cd ~/certs
mkcert -cert-file app.pem -key-file app-key.pem app.team1.test api.team1.test
mkcert -CAROOT                                # directory containing rootCA.pem
```

Procedure – Option 2: OpenSSL local CA (best for explaining the chain)

```

mkdir -p ~/certs && cd ~/certs
# 1. Team CA
openssl genrsa -out ca.key 4096
openssl req -x509 -new -nodes -key ca.key -sha256 -days 365 \
  -subj "/CN=Team1 Local CA" -out ca.crt
# 2. Server key + CSR
openssl genrsa -out app.key 2048
openssl req -new -key app.key -subj "/CN=app.team1.test" -out app.csr
# 3. Sign with SAN (browsers/curl require SAN, CN alone is not enough)
cat > san.ext <<EOF
subjectAltName=DNS:app.team1.test,DNS:api.team1.test
basicConstraints=CA:FALSE
keyUsage=digitalSignature,keyEncipherment
extendedKeyUsage=serverAuth
EOF
openssl x509 -req -in app.csr -CA ca.crt -CAkey ca.key -CAcreateserial \
  -out app.crt -days 365 -sha256 -extfile san.ext
# 4. Inspect
openssl x509 -in app.crt -noout -text | grep -A1 "Subject Alternative"
openssl verify -CAfile ca.crt app.crt

```

Trust the CA on every client Mac

```

# copy ca.crt (or rootCA.pem) to the client, then:
sudo security add-trusted-cert -d -r trustRoot \
  -k /Library/Keychains/System.keychain ca.crt
# GUI alternative: Keychain Access -> import -> double-click -> Trust: Always Trust
# Test WITHOUT -k:
curl -v https://app.team1.test:8443/
# If you use Homebrew's curl (OpenSSL) instead of /usr/bin/curl, pass --cacert ca.crt

```

Verify the handshake

```

openssl s_client -connect app.team1.test:8443 -servername app.team1.test -showcerts </dev/null
curl -vI https://app.team1.test:8443/ # shows TLS version, cipher, cert chain, verify
    result
curl --http2 -sI https://app.team1.test:8443/ | head -1 # HTTP/2 200 if enabled

```

Note

The final demonstration must **not** use `curl -k`. Certificate validity must be ≤ 825 days on macOS. The certificate names must match exactly the hostname typed (`app.team1.test`). **TLS 1.3 encrypts the Certificate message**, so it is not visible in Wireshark; to show the Certificate packet capture one request forced to TLS 1.2 (Task G).

Evidence to collect

Certificate setup notes; `openssl x509 -text` output (SAN visible); `curl -v` with “SSL certificate verify ok”; `openssl s_client` chain; browser padlock/certificate viewer; **no** `-k` flag anywhere in the demo.

Insert screenshot: curl -v output: TLS version, cipher, verify ok

Figure 3.6: Task E: curl -v showing successful certificate verification

Insert screenshot: browser certificate viewer showing issuer and SAN

Figure 3.7: Task E: browser padlock with certificate details

Be ready to explain (viva)

Walk through ClientHello → ServerHello → Certificate → Key Exchange → Finished. What is a CA and a chain of trust? What is SNI? Why do we install the CA certificate on clients? What does “TLS termination” mean? Why is edge-backend traffic unencrypted here and what are the risks? What changes in TLS 1.3 vs. 1.2?

3.7 Task F – Demonstrate HTTP Caching Behaviour

Theory

HTTP caching avoids repeating work. Three situations must be distinguished:

- **Fresh cache hit** – the stored copy is within `max-age`; the client reuses it *without any network request*.
- **Conditional request** – the copy is stale (or must be revalidated); the client sends `If-None-Match: <ETag>`. The server replies `304 Not Modified` with *no body*, saving bandwidth.
- **Full new request** – no usable copy, or content changed; the server returns `200 OK` with the full body.

CDNs apply the same mechanism at scale by keeping copies at edge locations close to users.

Procedure

```
# 1. Show cache headers
curl -I https://app.team1.test:8443/api/catalog
# Cache-Control: max-age=60
# ETag: "xxxxxxxx"

# 2. Conditional request -> expect 304
ETAG=$(curl -sI https://app.team1.test:8443/api/catalog | awk -F': ' 'tolower($1)=="etag"{print $2}' | tr -d '\r')
curl -I -H "If-None-Match: $ETAG" https://app.team1.test:8443/api/catalog
# HTTP/2 304 (or HTTP/1.1 304 Not Modified)
```

Browser: open DevTools → Network, load the URL, reload normally (size shows “memory cache”/“disk cache” within 60 s), then reload after 60 s to see a 304. A hard reload (Cmd+Shift+R) bypasses the cache.

Note

`curl` does not cache between invocations, so use the conditional request for the 304 demo and the browser for the true cache-hit demo. Both backends return *identical* bodies (hence identical ETags) for `/api/catalog`; only the `X-Backend` header differs. If the ETag differed between A and B, revalidations that landed on the other backend would return 200 instead of 304.

Evidence to collect

`curl -I` showing `Cache-Control`/ETag; the conditional request returning 304; DevTools screenshot of a cache hit.

Insert screenshot: curl -I output

Figure 3.8: Task F: Cache-Control and ETag headers via curl -I

Insert screenshot: curl -I -H If-None-Match output

Figure 3.9: Task F: conditional request returning 304 Not Modified

Insert screenshot: Network tab with memory/disk cache

Figure 3.10: Task F: browser DevTools showing cached response

Be ready to explain (viva)

Fresh hit vs. conditional request vs. full request. What do `max-age`, `no-store` and `ETag` mean? How do CDNs use caching? Why do dynamic endpoints (`/api/status`) use `no-store`?

3.8 Task G – Capture the Complete Protocol Flow

Procedure

1. On the client Mac, start Wireshark on the active interface (`en0`). Use capture filter `host <MAC1_IP>` or `host <MAC2_IP>` or capture everything and filter later.
2. Clear the DNS cache so a real query appears, then make one fresh request:

```
sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder
curl -v https://app.team1.test:8443/api/status
```

3. Repeat requests 4–6 times to capture alternating backends:

```
for i in 1 2 3 4; do curl -sI https://app.team1.test:8443/api/status | grep -i x-backend;
done
```

4. (To show the Certificate packet) force TLS 1.2 for one capture:

```
curl -v --tlsv1.2 --tls-max 1.2 https://app.team1.test:8443/
```

5. (Optional – decrypt HTTP) export TLS keys and load them in Wireshark (Preferences → Protocols → TLS → (Pre)-Master-Secret log):

```
export SSLKEYLOGFILE=$HOME/tlskeys.log
curl -s https://app.team1.test:8443/api/status      # Homebrew curl / Chrome honour this
variable
```

6. (Optional – prove TLS termination) capture on Mac 2 on the backend leg: `sudo tcpdump -i en0 -w backend-leg.pcap port 3001` or `port 3002`. HTTP is visible in clear text.
7. Save the final capture as `phase1.pcapng` (and `tls12.pcapng`).

What to point at in the capture

Event	Wireshark display filter	What to show and explain
DNS	<code>dns</code>	Query for <code>app.team1.test</code> and response with Mac 2's IP; UDP, client ephemeral port → 53; TTL in the answer.
TCP handshake	<code>tcp.flags.syn==1 && tcp.port==8443</code>	SYN → SYN-ACK → ACK; source/destination ports; initial sequence numbers; ACK = ISN+1.
TLS handshake	<code>tls.handshake</code>	ClientHello (SNI), ServerHello, (Certificate in TLS 1.2), ChangeCipherSpec/Finished; then Application Data.
Encrypted data	<code>tls.record.content_type!=23</code>	Payload unreadable: HTTP is inside TLS records.
HTTP headers	<code>curl -v</code> or decrypted capture	Request/response headers, status, <code>X-Backend</code> , <code>Cache-Control</code> .
Load balancing	repeated <code>curl</code>	<code>X-Backend</code> alternates A/B.
Ports	Statistics → Conversations	Client ephemeral port vs. well-known port per layer: 53/UDP (DNS), 8443/TCP (HTTPS).

Transport-layer notes for the viva

- **Socket pair / 4-tuple:** (client IP, client ephemeral port, edge IP, 8443) uniquely identifies the connection.
- **Reliability:** sequence numbers number bytes, ACKs confirm receipt, lost segments are re-transmitted after a timeout or duplicate ACKs, checksums detect corruption.
- **Flow control:** the receiver advertises a window size (visible in the TCP header) so the sender cannot overrun its buffer.
- **TCP vs. UDP:** TCP is connection-oriented and reliable; UDP is connectionless (DNS queries, QUIC/HTTP/3).

Evidence to collect

`phase1.pcapng` (+ `tls12.pcapng`); annotated screenshots of DNS, TCP handshake, TLS handshake, encrypted application data and ports.

Insert screenshot: Wireshark filter dns

Figure 3.11: Task G: DNS query and response (UDP/53)

Insert screenshot: Wireshark filter tcp.flags.syn==1

Figure 3.12: Task G: TCP three-way handshake (SYN, SYN-ACK, ACK)

Insert screenshot: Wireshark filter tls.handshake

Figure 3.13: Task G: TLS handshake packets

Insert screenshot: TLSv1.3 Application Data records

Figure 3.14: Task G: encrypted application data

Be ready to explain (viva)

Point to each packet and name its layer. Why is the payload unreadable? Why does the client use an ephemeral source port? What do sequence/ACK numbers prove? Why does DNS use UDP? Where does the SYN packet get its destination MAC?

3.9 Explanation-Only Topics

HTTP/2 and HTTP/3. HTTP/1.1 uses one request at a time per connection (text headers). HTTP/2 multiplexes many streams over one TCP+TLS connection with binary framing and header compression (HPACK), but TCP head-of-line blocking remains. HTTP/3 runs over QUIC (UDP/443) with TLS 1.3 built in, removing TCP head-of-line blocking and enabling faster connection setup. HTTP/3 is explanation-only here; HTTP/2 is shown with `curl -http2`.

Email protocols (explanation-only). SMTP (25/587; 465 with TLS) sends/relays mail; IMAP (143/993) keeps mail on the server and synchronises folders; POP3 (110/995) downloads mail to the client. Mail routing uses DNS MX records.

CDNs. A CDN replicates cacheable content at edge nodes near users, using DNS/anycast to direct clients and cache headers (**Cache-Control**, **ETag**) to decide freshness. Our nginx edge plays the role of a single edge node.

Chapter 4

Phase 1 – Required Failure Demonstrations

Each failure is triggered deliberately, observed with tools, and explained. Record the commands, the observed output and the explanation.

4.1 Failure 1 – Wrong DNS Server on a Client

Action: set a client's DNS server to an unused address (e.g. 192.168.1.250).

```
networksetup -setdnsservers Wi-Fi 192.168.1.250
dig app.team1.test          # -> timed out / no servers could be reached
ping -c 3 <MAC2_IP>         # -> still works (IP layer unaffected)
curl -sI https://<MAC2_IP>:8443/ -H "Host: app.team1.test" --cacert ca.crt # IP-level
                             connectivity proof (certificate name mismatch expected)
networksetup -setdnsservers Wi-Fi <MAC1_IP>      # restore
```

Observation / why: name lookup fails but direct IP connectivity still works. DNS and IP are independent layers; DNS only maps names to addresses.

4.2 Failure 2 – DNS Record Points to a Wrong IP

Action: edit dnsmasq so `app.team1.test` resolves to Mac 4 (or an unused IP), restart dnsmasq, flush the client cache.

```
# Mac 1: address=/app.team1.test/<WRONG_IP>
sudo brew services restart dnsmasq
# Client:
sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder
dig +short app.team1.test      # resolves successfully -> wrong IP
curl -v https://app.team1.test:8443/ # connection refused / timeout / certificate or content
                                     mismatch
```

Observation / why: resolution *succeeds*, but the client is sent to the wrong destination. DNS is a directory, not a connection; it cannot tell whether the target is correct or alive. Restore the correct record afterwards.

4.3 Failure 3 – One Backend Stopped

```
# Mac 3: stop Backend A (Ctrl+C)
for i in $(seq 1 6); do curl -s -D - -o /dev/null https://app.team1.test:8443/api/status | grep -
i -E "HTTP/|x-backend"; done
```

Observation / why: all responses come from Backend B with status 200, as nginx marks A as failed (`max_fails/fail_timeout`) and uses `proxy_next_upstream` to retry the request on B. The first request after the failure may take up to `proxy_connect_timeout` seconds.

4.4 Failure 4 – Both Backends Stopped

```
# stop Backend A and Backend B
curl -v https://app.team1.test:8443/api/status
dig +short app.team1.test # DNS still fine
openssl s_client -connect app.team1.test:8443 -servername app.team1.test </dev/null | grep -i "
verify return" # TLS still fine
```

Observation / why: DNS and TLS still work (they end at the edge), but nginx returns 502 Bad Gateway, meaning the edge is healthy but its upstreams are not. It shows where the edge ends and the backend begins.

4.5 Failure 5 – Wrong Destination Port on the Client

```
ping -c 3 app.team1.test # host reachable
curl -v https://app.team1.test:9999/ # Connection refused (RST) or timeout
nc -vz <MAC2_IP> 9999 # failed to connect
```

Observation / why: the host is reachable at the IP layer, but nothing listens on that TCP port, so the host replies with RST (refused). Ports and IP addresses are separate identifiers. In Wireshark: SYN followed by RST, ACK.

4.6 Failure Summary Table

Failure	Layer affected	Symptom	What still works
Wrong DNS server	Application (DNS)	Name lookup fails	ping by IP
Wrong DNS record	Application (DNS)	Wrong destination reached	DNS answers normally
One backend down	Application (up-stream)	None for clients (failover)	HTTPS, DNS, B
Both backends down	Application (up-stream)	502 Bad Gateway	DNS, TCP, TLS to edge
Wrong port	Transport	Connection refused/-timeout	DNS, ICMP ping

Table 4.1: Failure scenarios and isolation by layer.

Insert screenshot: one screenshot per scenario, add more figures as needed

Figure 4.1: Failure demos: screenshots for all five scenarios

Chapter 5

Phase 2 – Harden, Recover and Troubleshoot

Success condition

Phase 2 extends the Phase 1 infrastructure (no rebuild). It is complete when backup DNS, TTL behaviour, service isolation, HA failover and at least one troubleshooting challenge are demonstrated.

5.1 Extension A – Backup DNS Resolver

1. On a Mac other than Mac 1 (e.g. Mac 3), install dnsmasq and use the *same records* (Appendix A.1, change `listen-address`).
2. Configure clients with both resolvers (primary first):

```
networksetup -setdnsservers Wi-Fi <MAC1_IP> <BACKUP_DNS_IP>
scutil --dns | grep nameserver
```

3. Stop the primary: `sudo brew services stop dnsmasq` on Mac 1.
4. From the client:

```
dig +short app.team1.test @<BACKUP_DNS_IP>          # backup answers
dig +tries=1 +time=2 app.team1.test @<MAC1_IP>       # primary: no response
curl -sI https://app.team1.test:8443/ | head -1     # system resolver falls back; may add a
few seconds delay
```

5. Restart the primary afterwards.

Explain: A *DNS service failure* stops name resolution (clients cannot find any IP) whereas an *application server failure* leaves resolution working but the service itself unreachable or returning errors. The resolver library tries nameservers in order and moves to the next after a timeout, so failover costs a short delay.

Insert screenshot: dig / curl output with primary stopped

Figure 5.1: Extension A: resolution via backup DNS while primary is stopped

5.2 Extension B – DNS TTL and Controlled Record Change

1. Give the record a short, visible TTL (30 s) using `host-record` (it accepts a TTL):

```
# dnsmasq.conf on Mac 1 (replace the address= lines for app.team1.test)
host-record=app.team1.test,<MAC2_IP>,30
```

2. Resolve from a client through the *system resolver*, printing the time:

```
dig app.team1.test +noall +answer          # TTL column = 30
while true; do date +%T; dscacheutil -q host -a name app.team1.test | grep ip_address;
sleep 5; done
```

3. On Mac 1 change the record to a different IP (e.g. Mac 4) and `sudo brew services restart dnsmasq`.
4. Observe: the client keeps returning the **old** answer until the TTL expires, then switches to the new one.
5. Flush the cache and observe the **immediate** change:

```
sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder
```

6. Restore the original record and TTL.

Note

`dig` queries the server directly and therefore sees changes immediately – use the system-resolver loop (or `ping/curl`) to demonstrate caching. Real caching behaviour of mDNSResponder can vary; record exactly what you observe with timestamps.

Explain: The TTL tells resolvers how long they may cache an answer. Lowering the TTL *before* a planned migration (e.g. to 30 s) lets traffic cut over quickly; a long TTL speeds lookups and reduces load but delays cutover. Cloud DNS (Route 53) uses the same mechanism for blue/green migration and DNS failover.

Insert screenshot: timestamped loop output

Figure 5.2: Extension B: old answer cached, then new answer after TTL expiry, then after flush

5.3 Extension C – Service Isolation (Backend Firewall Rules)

Goal: only Mac 2 (the edge) may reach ports 3001/3002; other clients are blocked.

1. **Backup first** (rollback copy) on Mac 3 and Mac 4:

```
sudo cp /etc/pf.conf /etc/pf.conf.rollback
sudo pfctl -s rules > ~/pf-rules-before.txt
sudo pfctl -s info | head -3           # note whether pf is Enabled/Disabled
```

2. Create a rule file that keeps the default ruleset and appends isolation rules (Mac 3 shown; use port 3002 on Mac 4):

```
sudo cp /etc/pf.conf /etc/pf.cn.conf
sudo tee -a /etc/pf.cn.conf >/dev/null <<'EOF'
# --- CN project isolation: only the edge may reach the backend ---
pass in quick proto tcp from <MAC2_IP> to any port 3001
block drop in quick proto tcp from any to any port 3001
EOF
sudo pfctl -nf /etc/pf.cn.conf           # syntax check only
sudo pfctl -f /etc/pf.cn.conf -e        # load and enable
```

3. Demonstrate:

```
# From Mac 2 (edge) -> works
curl -i http://<MAC3_IP>:3001/api/status
# From a client Mac -> blocked (hangs/times out)
curl -m 5 -i http://<MAC3_IP>:3001/api/status      # timeout
nc -vz -w 3 <MAC3_IP> 3001
# Through the edge by name -> still works
curl -s https://app.team1.test:8443/api/status
```

4. **Rollback** after the demonstration:

```
sudo pfctl -f /etc/pf.conf.rollback       # restore original ruleset
sudo pfctl -d                             # disable pf if it was disabled before
sudo pfctl -s rules | diff - ~/pf-rules-before.txt && echo "rollback OK"
```

Explain: the firewall filters by source IP and destination port (L3/L4); a **drop** (silent) rule causes timeouts whereas **return** (reset) would cause immediate refusal. This implements *defence in depth* / least privilege: backends are private and the only entry point is the edge (similar to cloud security groups allowing only the load balancer). Note that source-IP rules can be spoofed on a shared LAN – they demonstrate the concept, not strong security.

Insert screenshot: three terminal outputs

Figure 5.3: Extension C: Mac 2 reaches backend, client Mac blocked, name still works

5.4 Extension D – High-Availability Failover Behaviour

1. Configure passive health checks and retries (see the Phase 2 nginx config in Appendix A.2):
`max_fails=2 fail_timeout=10s, proxy_connect_timeout 2s, proxy_next_upstream error timeout http_502 http_503.`

2. Stop Backend A; run the request loop; all responses must show **X-Backend:** B with status 200.
3. Restart Backend A; after `fail_timeout` nginx retries it and responses alternate A/B again.
4. Identify the single point of failure.

Single points of failure. The *edge nginx on Mac 2* is the main SPOF (if it dies, no client can reach any backend). Other SPOFs: the DNS server (reduced by Extension A), the LAN router/Wi-Fi, and the single TLS CA/cert. **Eliminating the edge SPOF:** run two (or more) edge nodes and either (a) use a floating/virtual IP with VRRP/keepalived, (b) publish multiple A records or health-checked DNS failover (Route 53), or (c) put an anycast/cloud load balancer in front. Extension E demonstrates the DNS-based option.

Note

Open-source nginx performs *passive* health checking (it learns of failure from failed real requests). Active probing (`health_check`) is an nginx Plus feature.

Insert screenshot: curl loop outputs

Figure 5.4: Extension D: Backend A stopped, all requests served by B; then A restored

5.5 Extension E – Controlled Edge Migration (DNS-Based Cutover)

1. Prepare a standby nginx on a spare Mac (Mac 3 or Mac 4) with the *same* nginx configuration and the *same* certificate/key (they are valid for the name, not the IP).
2. Lower the TTL (30s) in advance. Verify the standby: `curl -resolve app.team1.test:8443:<STANDBY_IP> https://app.team1.test:8443/api/status`.
3. Change the DNS record to the standby IP and restart dnsmasq.
4. Observe TTL effects: a client with a cached answer still reaches the old edge until the TTL expires; new/flushed lookups reach the standby. Prove which edge answered by adding `add_header X-Edge "mac2"; / "standby";` in each nginx config.

```
while true; do date +%T; curl -s -D - -o /dev/null https://app.team1.test:8443/api/status |
  grep -i -E "x-edge|x-backend"; sleep 5; done
```

5. Optionally stop nginx on Mac 2 after cutover and show the service still works through the standby.

Explain: this is how production traffic is migrated or failed over with DNS. The cutover is not instantaneous because resolvers cache answers until the TTL expires; hence the TTL is lowered before migration.

Insert screenshot: X-Edge header changing over time

Figure 5.5: Extension E: old edge answering, then standby after TTL expiry

5.6 Extension F – Faculty-Injected Troubleshooting Challenge

During evaluation, faculty inject one configuration fault. Diagnose **layer by layer, out loud**: DNS → TCP → TLS → Application. Partial credit is awarded for correct methodology even if the fix is incomplete.

Systematic diagnosis runbook

Step	Test (command)	Healthy result	If it fails, suspect
0. LAN/IP	<code>ping <MAC2_IP>; <gateway></code>	<code>ping</code> Replies	Wi-Fi, wrong network, VPN, firewall blocking ICMP
1. DNS	<code>dig app.team1.test; scutil -dns</code>	Mac 2 IP, correct server, status NOERROR	Wrong client resolver, dns-masq down/misconfigured, wrong record, stale cache
2. TCP	<code>nc -vz <MAC2_IP> 8443; lsof -nP -iTCP:8443</code>	“succeeded” / LISTEN	nginx down, wrong port, firewall, wrong bind address
3. TLS	<code>openssl s_client -connect app.team1.test:8443 -servername app.team1.test</code>	“Verify return code: 0 (ok)”	Expired cert, SAN/name mismatch, CA not trusted, wrong key/cert pair
4. HTTP/App	<code>curl -vi https://app.team1.test:8443/api/status</code>	200 + X-Backend	502/504 (upstream down, wrong upstream port/IP), 404, nginx config
5. Back-ends	<code>curl -i http://<MAC3_IP>:3001/api/status</code>	200	App stopped, bound to 127.0.0.1, pf rule blocking
6. Logs	<code>tail -f \$(brew -prefix)/var/log/nginx/error.log; tail /tmp/dnsmasq.log</code>	Clean	Shows the failing component

Likely injected faults and their layer

Fault	Layer	Tell-tale symptom
dnsmasq stopped / wrong client DNS	DNS	dig timeout; ping by IP works
Wrong A record	DNS data	Resolves to unexpected IP; wrong server or refused
nginx stopped / wrong listen port	TCP	Connection refused on 8443
Firewall rule blocking edge port	TCP	Timeout (no RST)
Certificate expired, wrong SAN, wrong key	TLS	curl : certificate verify failed / name mismatch
Wrong <code>proxy_pass</code> /upstream port	Application	502 Bad Gateway, connect() failed in error log
Backend bound to 127.0.0.1	TCP (backend)	Reachable locally, refused remotely; edge returns 502
nginx syntax error after edit	Application/edge	nginx -t fails; reload keeps old config

Table 5.2: Fault catalogue to practise before the evaluation.

Chapter 6

Final Demonstration (11-Step Sequence)

The evaluator can interrupt at any step. All members must be able to run and explain every step.

#	Step	What to run / show	Key explanation
1	Topology and inventory	Figure 2.1, Tables 2.2 and 2.3, request-flow Figure 2.2	Role of each Mac, cloud equivalents.
2	Confirm private LAN	ping between all machines; ipconfig getifaddr en0 on each	Same subnet, ARP, no router needed.
3	Resolve domain from client	dig app.team1.test (SERVER = Mac 1, ANSWER = Mac 2)	DNS finds the IP; it does not connect.
4	Open service over HTTPS	Browser/curl https://app.team1.test:8443/ - no warnings, no IP, no -k	Chain of trust, SAN, TLS termination.
5	Load balancing	6× curl -sI ... grep -i x-backend	Round-robin at L7; client unaware of backends.
6	Wireshark evidence	Open phase1.pcapng: SYN/SYN-ACK/ACK, TLS handshake	Layers, ports, encrypted payload.
7	Headers and caching	curl -I (Cache-Control/ETag), conditional 304 or browser cache hit	Fresh hit vs. conditional vs. full.
8	Fail one backend	Stop Mac 3 backend; repeat requests	Passive health check, failover to B only.
9	Phase 2 resilience	Backup DNS failover and/or TTL/cutover (Ext. A/B/E)	DNS vs. application failure; TTL caching.
10	Diagnose injected fault	Runbook: DNS → TCP → TLS → App	Speak the method out loud.
11	Individual viva	Each member answers questions without notes	End-to-end understanding.

6.1 Pre-Demo Readiness Checklist

- ☐ All four Macs on the same Wi-Fi; IPs unchanged (consider DHCP reservations or static IPs; re-check addresses on demo day).
- ☐ Clients' DNS = Mac 1 (+ backup); dnsmasq, nginx and both backends running; `sudo nginx -t` OK.
- ☐ CA certificate trusted on every client; no `-k` anywhere; certificate not expired.
- ☐ Wireshark captures saved and filters prepared; screenshots in the evidence folder.
- ☐ pf rules ready with rollback copy; standby nginx ready; backup dnsmasq ready.
- ☐ macOS Firewall allows incoming connections for nginx, dnsmasq, Python; Macs set not to sleep (`caffeinate -d`).
- ☐ Terminal windows pre-arranged; command cheat sheet (Appendix B) printed.

Chapter 7

Deliverables, Evidence Index and Marks

7.1 Deliverables

Deliverable	Due	Required contents (and where in this document)
Architecture Document	Phase 1 (updated for Phase 2)	Topology, roles/IP/service table, request-flow diagram (Chapter 2)
Configuration Bundle	Phase 1 + 2 additions	dnsmasq, nginx (Phase 1+2), TLS setup notes, backend launch steps, firewall rules (Appendices A.1–A.3, Task E, Ext. C)
Backend Code	Phase 1	Complete code for both backends + helper scripts on GitHub or zip (Appendix A.3)
Evidence Folder	Phase 1 + 2 additions	dig/nslookup, curl/browser headers, Wireshark (DNS, TCP, TLS), caching demo, all failure demos (Table 7.2)
Phase 1 Checkpoint	Phase 1 review	Working live system + structured demo; everyone can explain any component
Phase 2 Final Report	Phase 2 review	What changed, resilience tests and results, troubleshooting findings, one paragraph per extension (Chapter 9)
Final Presentation	Final evaluation	Live demo following the 11 steps; every member participates

7.2 Suggested Repository Layout

```
cn-project-team1/  
|-- README.md  
|-- docs/                # this PDF, topology.png, flow.png  
|-- backend/  
|   |-- backend.py  
|   |-- requirements.txt  
|   '-- run_backend.sh  
-- config/  
|   |-- dnsmasq-primary.conf  
|   |-- dnsmasq-backup.conf  
|   |-- nginx-phase1.conf
```

```
| |-- nginx-phase2.conf
| |-- pf.cn.conf
| '-- cert-setup-notes.md
'-- evidence/
    |-- phase1/ (screenshots, phase1.pcapng, tls12.pcapng)
    |-- failures/
    '-- phase2/
```

Do **not** commit private keys (*.key, app-key.pem, ca.key) to a public repository.

7.3 Evidence Index (30-second rule)

ID	Task	Evidence file / description	Done
E1	A	ifconfig/ipconfig output on every Mac	☐
E2	A	Ping matrix screenshots	☐
E3	A	Topology diagram	☐
E4	B	dnsmasq.conf + dig/nslookup from two clients	☐
E5	C	Backend curl -i showing X-Backend (both)	☐
E6	D	nginx.conf + 6-request alternating output	☐
E7	E	Cert setup, openssl text, curl -v verify ok, padlock	☐
E8	F	Cache-Control/ETag, 304, DevTools cache hit	☐
E9	G	phase1.pcapng, tls12.pcapng, annotated screenshots	☐
E10	P1 failures	Five failure demos (commands + output + explanation)	☐
E11	Ext. A	Backup DNS failover	☐
E12	Ext. B	TTL timeline + flush	☐
E13	Ext. C	pf rules, blocked/allowed proof, rollback proof	☐
E14	Ext. D	One backend down/restored, SPOF analysis	☐
E15	Ext. E	Standby edge cutover with X-Edge timeline	☐
E16	Ext. F	Troubleshooting practice notes	☐

Table 7.2: Evidence index.

7.4 Marks Breakdown and Self-Assessment

Review	Area	Notes	Marks	Ready
1	LAN setup + private DNS (Tasks A+B)	Topology, ping, DNS records, client resolvers	10	☐
1	Backends + reverse proxy + LB (C+D)	Both running, nginx upstream, X-Backend visible	10	☐
1	HTTPS/TLS correctness and explanation (E)	Cert setup, nginx TLS, handshake explained	8	☐
1	Packet analysis and protocol flow (G)	DNS, TCP, TLS captures, ports	7	☐
1	HTTP caching + transport understanding (F)	Cache-Control, 304/cache hit	5	☐
1	Individual viva, Phase 1 concepts	Per student	10	☐
Review 1 total (40 team + 10 viva)			50	
2	Resilience and HA failover (Ext. A, B, D)	Backup DNS, TTL, HA failover	15	☐
2	Service isolation + edge migration (C, E)	Firewall rules, DNS cutover, TTL effects	10	☐
2	Troubleshooting challenge (F)	Correct layer, systematic diagnosis	10	☐
2	Final report quality + documentation	Changes, test results, learning summary	5	☐
2	Individual viva, Phase 2 + full system	Per student, end to end	10	☐
Review 2 total (40 team + 10 viva)			50	
TOTAL			100	

Table 7.3: Marks breakdown from the course brief.

Chapter 8

Viva Preparation

Answer from understanding, not from notes. Practise these as a team; each member should be able to answer any of them.

8.1 Phase 1 Concepts

Q1. Trace a request for `https://app.team1.test:8443/api/status` from keypress to response.

Resolver asks Mac 1 (UDP/53) → gets Mac 2's IP → TCP three-way handshake to Mac 2:8443 → TLS handshake (certificate validated against the team CA, SAN matched) → HTTP request inside TLS → nginx decrypts, picks a backend round-robin, forwards over HTTP → backend responds with JSON and **X-Backend** → nginx re-encrypts and returns it.

Q2. Difference between DNS resolution and the connection?

DNS only maps a name to an IP (a directory lookup, UDP/53, cached for the TTL). The TCP/TLS/HTTP connection is a separate exchange with the resolved address.

Q3. Why is DNS usually UDP?

Small single request/response, no handshake overhead; the application retries on loss. TCP is used for large responses (truncation), zone transfers and DNS-over-TLS.

Q4. Explain the TCP three-way handshake and what sequence/ACK numbers do.

SYN (client ISN) → SYN-ACK (server ISN, ACK = client ISN+1) → ACK (ACK = server ISN+1). Sequence numbers identify bytes; ACKs confirm receipt; gaps trigger retransmission, giving reliable ordered delivery.

Q5. How does TCP flow control differ from congestion control?

Flow control protects the *receiver* using the advertised window; congestion control protects the *network* using the congestion window (slow start, AIMD).

Q6. What is a socket / ephemeral port / well-known port?

A socket is IP:port (plus protocol); a connection is identified by the 4-tuple. Clients pick ephemeral ports (49152–65535 on macOS) while servers listen on fixed ports (53, 443/8443, 3001).

Q7. Walk through the TLS handshake. Why can Wireshark not read the HTTP data?

See Task E. Application data is encrypted with keys derived from the (EC)DHE exchange that Wireshark does not have unless the key log file is supplied. In TLS 1.3 even the Certificate message is encrypted.

Q8. What does the certificate prove and how does the client validate it?

It binds the hostname (SAN) to a public key, signed by a CA. The client checks the signature

chain up to a trusted root, validity dates, hostname match and key usage; then the server proves possession of the private key in the handshake.

Q9. Why is TLS terminated at the edge? Downsides?

Central certificate management, offloaded crypto, ability to inspect/route HTTP. Downside: edge-backend leg is unencrypted unless re-encrypted (mTLS/HTTPS upstream).

Q10. Why does the client never need backend IPs?

Names point to the edge; the edge is the only public entry and maps requests to upstreams, allowing backends to be added, removed or moved transparently.

Q11. Round-robin vs. least_conn vs. ip_hash.

Rotation in turn / fewest active connections / sticky by client IP hash. Round-robin is simplest and best for similar, short requests.

Q12. Fresh cache hit vs. conditional request vs. full request.

No network / network round-trip returning 304 without body / full 200 with body.

Q13. OSI vs. TCP/IP – where do DNS, HTTP, TLS, TCP, IP and Ethernet sit?

See Section 2.7.

Q14. What does .test mean and why not .local?

.test is reserved for testing (RFC 6761) and never resolves publicly; .local is used by multicast DNS (mDNS/Bonjour) and conflicts with unicast lookups on macOS.

Q15. Map each component to the cloud.

dnsmasq = Route 53; nginx = ALB/CDN edge; backends = EC2/containers; pf rules = security groups; DNS cutover = Route 53 weighted/failover routing.

Q16. Explain HTTP/1.1 vs. HTTP/2 vs. HTTP/3; and SMTP/IMAP/POP3.

See Section 3.9.

8.2 Phase 2 and Whole-System Concepts

Q17. DNS service failure vs. application server failure.

DNS failure: no name resolution (NXDOMAIN/timeout) even though servers are up. Application failure: resolution succeeds but the connection fails or returns 5xx.

Q18. How does the client fail over between DNS servers?

The resolver tries the first nameserver, times out, and moves to the next; failover costs a delay and is not load balancing.

Q19. What is TTL and why does the old answer persist after a record change?

Caches may keep an answer for TTL seconds; nothing notifies them of changes. Flushing the cache or waiting for expiry shows the update.

Q20. How does lowering TTL help migration?

Lower it ahead of time so the old answer expires quickly after cutover; raise it again afterwards to reduce query load.

Q21. How do the pf rules isolate the backends? What is the difference between drop and reject?

Allow from the edge IP, block all other sources to ports 3001/3002. **drop** = silent (timeout); **return** = RST/ICMP (fast failure).

Q22. How does nginx detect a failed backend? What are the limits?

Passive checks: failed connections/timeouts count toward **max_fails**; the server is skipped for **fail_timeout**. Limitation: real requests detect failures (first request may be slow); active checks require nginx Plus or external tools.

Q23. What is the single point of failure and how would you remove it?

Edge nginx (also DNS and the LAN). Add a second edge with keepalived/VIP or health-checked DNS, redundant DNS servers, and redundant network paths.

Q24. Describe your troubleshooting order and why.

Bottom-up/inside-out: IP reachability → DNS → TCP port → TLS → HTTP/app → backend, because each layer depends on the one beneath it; the first failing test localises the fault.

Q25. Why would curl give 502 but openssl s_client succeed?

TLS terminates at the edge and works; the failure is between edge and upstream (backend down/misconfigured).

Chapter 9

Phase 2 Final Report

9.1 What Changed from Phase 1

Component	Phase 1	Phase 2 change
DNS	Single dnsmasq on Mac 1	Backup dnsmasq on [TODO: Mac] ; clients list two resolvers; 30s TTL for tests
nginx edge	Round-robin, TLS	Passive health checks, retries, X-Edge header; standby edge prepared
Backends	Reachable from any LAN host	Reachable only from Mac 2 (pf rules, with rollback)
Operations	Manual checks	Layered troubleshooting runbook

Table 9.1: Differences between Phase 1 and Phase 2.

9.2 Resilience Test Results

Test	Action	Observed result	Pass/Fail
Backup DNS	Stop dnsmasq on Mac 1	[TODO: observed]	[TODO:]
TTL change	Change record, wait 30s, flush	[TODO: observed timeline]	[TODO:]
Service isolation	Client → :3001 vs. Mac 2 → :3001	[TODO: blocked / allowed]	[TODO:]
HA failover	Stop Backend A, then restore	[TODO: only B, then A+B]	[TODO:]
Edge migration	Point DNS to standby edge	[TODO: X-Edge change time]	[TODO:]
Injected fault	[TODO: fault introduced by faculty]	[TODO: layer + fix]	[TODO:]

Table 9.2: Phase 2 test log (complete during rehearsal).

9.3 Troubleshooting Findings

[TODO: Describe the fault(s) practised, the layer affected, the commands used, time to diagnose, and the fix.]

9.4 Learning Summary (one paragraph per extension)

Extension A – Backup DNS. Running a second dnsmasq with identical records removed the DNS server as a single point of failure for name resolution. When the primary was stopped, clients timed out on the first nameserver and then succeeded via the backup, showing that DNS availability is independent of application availability and that resolver failover costs latency but not correctness. [TODO: add observed delay]

Extension B – TTL and controlled record change. With a 30 s TTL, the client continued receiving the old address after the record was changed until the cache entry expired, and received the new address immediately after a manual flush. This demonstrated that DNS changes propagate by expiry rather than push, and why production teams lower the TTL before migrations and raise it afterwards. [TODO: add timestamps]

Extension C – Service isolation. pf rules allowed only Mac 2 to reach the backend ports. Direct client connections timed out while the same backends remained reachable through the edge by name, illustrating least privilege and the role of a single controlled entry point, analogous to cloud security groups. The original ruleset was restored from the rollback copy.

Extension D – HA failover. With `max_fails`, `fail_timeout` and `proxy_next_upstream`, stopping Backend A left the service available through Backend B, and restarting A restored alternating distribution. The edge nginx remained the single point of failure; eliminating it requires redundant edge nodes with a floating IP or health-checked DNS.

Extension E – Edge migration. Pointing the DNS record at a standby nginx moved traffic between edges without client changes. Clients with cached answers briefly kept using the old edge until the TTL expired, matching real DNS-based cutovers and blue/green migrations.

Extension F – Troubleshooting. Following DNS → TCP → TLS → application and checking logs at each stage allowed faults to be localised quickly, because each layer depends on those below it. The runbook in Chapter 5 turns this into repeatable practice.

9.5 Conclusion

The project demonstrated that a single HTTPS request involves cooperating protocols at every layer – DNS to find the edge, TCP for reliable transport, TLS for authenticated encryption, HTTP for application semantics, and a reverse proxy/load balancer for scale and availability – and that each can be observed, broken and repaired independently. The most important skill gained is layered diagnosis: identifying whether a fault is in naming, transport, security or application before attempting a fix.

Appendix A

Configuration Files

A.1 dnsmasq – Primary (Mac 1)

File: `$(brew -prefix)/etc/dnsmasq.conf`

```
# ---- CN project: primary private DNS (Mac 1) ----
port=53
listen-address=<MAC1_IP>,127.0.0.1
bind-interfaces

# Only answer for our private zone; do not forward .test upstream
no-resolv
local=/team1.test/

# Phase 1 records
address=/app.team1.test/<MAC2_IP>
address=/api.team1.test/<MAC2_IP>

# Phase 2 / Extension B (use INSTEAD of address= for a visible TTL of 30 s):
# host-record=app.team1.test,<MAC2_IP>,30
# host-record=api.team1.test,<MAC2_IP>,30

# Evidence logging
log-queries
log-facility=/tmp/dnsmasq.log
```

Backup resolver (Extension A): identical file, but `listen-address=<BACKUP_DNS_IP>,127.0.0.1`. Records on both must be changed together during experiments.

Note

If clients should also reach the Internet while using Mac 1 as DNS, remove `no-resolv` and add `server=1.1.1.1`, or list a public resolver as a third DNS server on the client. Restart after edits: `sudo brew services restart dnsmasq`.

A.2 nginx – Phase 1 and Phase 2 (Mac 2)

File: `$(brew -prefix)/etc/nginx/nginx.conf` (or an included `servers/cn.conf`). Needs nginx $\geq 1.25.1$ for the `http2 on`; directive; on older versions use `listen 8443 ssl http2`;

```
worker_processes 1;
events { worker_connections 1024; }
```

```

http {
    include      mime.types;
    default_type application/octet-stream;
    sendfile     on;

    log_format cn '$remote_addr [$time_local] "$request" $status '
                  'backend=$upstream_addr xb=$upstream_http_x_backend rt=$request_time';
    access_log /opt/homebrew/var/log/nginx/access.log cn;
    error_log  /opt/homebrew/var/log/nginx/error.log warn;

    # ----- Upstream pool (Phase 1 round-robin + Phase 2 passive health) -----
    upstream backends {
        # least_conn;                                     # alternative strategy
        server <MAC3_IP>:3001 max_fails=2 fail_timeout=10s;
        server <MAC4_IP>:3002 max_fails=2 fail_timeout=10s;
    }

    # ----- HTTP -> HTTPS redirect (optional) -----
    server {
        listen 8080;
        server_name app.team1.test api.team1.test;
        return 301 https://$host:8443$request_uri;
    }

    # ----- HTTPS edge -----
    server {
        listen 8443 ssl;
        http2 on;
        server_name app.team1.test api.team1.test;

        ssl_certificate      /Users/<user>/certs/app.crt;          # or app.pem (mkcert)
        ssl_certificate_key  /Users/<user>/certs/app.key;          # or app-key.pem
        ssl_protocols        TLSv1.2 TLSv1.3;
        ssl_session_cache    shared:SSL:5m;
        ssl_session_timeout  10m;

        add_header X-Edge "mac2" always;                          # change to "standby" on the
        standby edge

        location / {
            proxy_pass http://backends;
            proxy_http_version 1.1;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;

            # ---- Phase 2: failover behaviour ----
            proxy_connect_timeout 2s;
            proxy_read_timeout 10s;
            proxy_next_upstream error timeout http_502 http_503 http_504;
            proxy_next_upstream_tries 2;
        }
    }
}

```

Path note: Apple-silicon Homebrew uses `/opt/homebrew`; Intel Macs use `/usr/local`. Use `brew -prefix` to confirm. X-Backend is passed through from the upstream response automatically.

A.3 Backend Application (Mac 3 and Mac 4)

File: backend.py (identical on both Macs; only environment variables differ).

```
#!/usr/bin/env python3
"""Minimal REST backend for the CN project. The network is the project, not the app."""
import os, time, hashlib
from flask import Flask, jsonify, request, make_response

ID = os.environ.get("BACKEND_ID", "A") # "A" or "B"
PORT = int(os.environ.get("BACKEND_PORT", "3001")) # 3001 (A) / 3002 (B)

app = Flask(__name__)

@app.after_request
def add_backend_header(resp):
    resp.headers["X-Backend"] = ID # proves which backend answered
    return resp

@app.get("/")
def index():
    return jsonify(message="Backend %s is running" % ID, port=PORT)

@app.get("/api/status")
def status():
    resp = jsonify(backend=ID, status="ok", time=int(time.time()))
    resp.headers["Cache-Control"] = "no-store" # dynamic endpoint: never cache
    return resp

@app.get("/api/catalog")
def catalog():
    # Identical body on A and B => identical ETag => 304 works whichever backend answers
    resp = make_response(jsonify(resource="catalog", version=1,
                                items=["router", "switch", "firewall"]))
    resp.set_etag(hashlib.sha256(resp.get_data()).hexdigest()[:16])
    resp.headers["Cache-Control"] = "max-age=60"
    return resp.make_conditional(request) # returns 304 when If-None-Match matches

@app.get("/health")
def health():
    return jsonify(status="up", backend=ID)

if __name__ == "__main__":
    # 0.0.0.0 => reachable from other Macs on the LAN (NOT 127.0.0.1)
    app.run(host="0.0.0.0", port=PORT)
```

```
# run_backend.sh
#!/usr/bin/env bash
cd "$(dirname "$0")"
[ -d venv ] || python3 -m venv venv
source venv/bin/activate
pip install -q flask
BACKEND_ID="${1:-A}" BACKEND_PORT="${2:-3001}" python backend.py
# usage: ./run_backend.sh A 3001 (Mac 3) ./run_backend.sh B 3002 (Mac 4)
```

A.4 pf Rules (Extension C)

```
# /etc/pf.cn.conf = copy of /etc/pf.conf + the following lines (Mac 3; use port 3002 on Mac 4)
pass in quick proto tcp from <MAC2_IP> to any port 3001
block drop in quick proto tcp from any to any port 3001
```

Appendix B

Command Cheat Sheet

Purpose	Command
My IP / gateway / MAC	<code>ipconfig getifaddr en0;</code> <code>netstat -nr grep default;</code> <code>ifconfig en0 grep ether</code>
Resolver in use	<code>scutil -dns; networksetup -getdnsservers Wi-Fi</code>
Set / reset DNS	<code>networksetup -setdnsservers Wi-Fi <ip1> <ip2>;</code> <code>networksetup -setdnsservers Wi-Fi empty</code>
DNS query	<code>dig app.team1.test;</code> <code>dig +short app.team1.test @<ip>;</code> <code>nslookup app.team1.test</code>
System-resolver lookup	<code>dscacheutil -q host -a name app.team1.test</code>
Flush DNS cache	<code>sudo dscacheutil -flushcache; sudo killall -HUP</code> <code>mDNSResponder</code>
Start/stop/restart dnsmasq	<code>sudo brew services start stop restart dnsmasq</code>
Check listeners	<code>sudo lsof -nP -iTCP -sTCP:LISTEN;</code> <code>sudo lsof -nP</code> <code>-iUDP:53</code>
nginx control	<code>sudo nginx -t; sudo nginx; sudo nginx -s reload; sudo</code> <code>nginx -s stop</code>
HTTPS request	<code>curl -v https://app.team1.test:8443/; curl -sI ...; curl</code> <code>-i ...</code>
Force TLS version / HTTP version	<code>curl -tlsv1.2 -tls-max 1.2 ...; curl -http2 -sI ...;</code> <code>curl -http1.1 -sI ...</code>
Bypass DNS for testing	<code>curl -resolve app.team1.test:8443:<ip></code> <code>https://app.team1.test:8443/</code>
Port test	<code>nc -vz <ip> <port></code>
Certificate inspection	<code>openssl s_client -connect host:8443 -servername host</code> <code>-showcerts; openssl x509 -in app.crt -noout -text</code>
Capture (CLI)	<code>sudo tcpdump -i en0 -w cap.pcap host <ip></code>
Keep Mac awake	<code>caffeinate -d</code>
pf	<code>sudo pfctl -s rules; sudo pfctl -nf file; sudo pfctl -f</code> <code>file -e; sudo pfctl -d</code>
Logs	<code>tail -f /opt/homebrew/var/log/nginx/error.log; tail -f</code> <code>/tmp/dnsmasq.log</code>

Appendix C

Wireshark Filter Reference

Display filter	Shows
<code>dns or udp.port==53</code>	DNS queries and responses
<code>dns.qry.name contains "team1"</code>	Queries for our domain only
<code>tcp.port==8443</code>	All HTTPS traffic to the edge
<code>tcp.flags.syn==1</code>	SYN and SYN-ACK packets (handshake start)
<code>tcp.flags.reset==1</code>	Connection refusals (wrong port demo)
<code>tls.handshake</code>	TLS handshake messages
<code>tls.handshake.type==1 / ==2 / ==11</code>	ClientHello / ServerHello / Certificate (TLS 1.2)
<code>tls.handshake.extensions_server_name</code>	SNI value
<code>tls.record.content_type==23</code>	Encrypted application data
<code>http</code> (after key-log decryption)	Decrypted HTTP requests/responses
<code>tcp.analysis.retransmission</code>	Retransmissions (reliability demo)
<code>ip.addr==<MAC2_IP> && tcp</code>	Everything between client and edge

Useful views: *Statistics* → *Conversations*, *Follow* → *TCP Stream*, *Statistics* → *Flow Graph*, and the packet-detail pane to show the encapsulation Ethernet → IP → TCP → TLS.

Appendix D

Glossary

A record	DNS record mapping a name to an IPv4 address.
CA	Certificate Authority; signs certificates that clients trust.
Ephemeral port	Temporary client-side port chosen for one connection.
Load balancer	Component distributing requests across several servers.
Passive health check	Detecting failed upstreams from failed real requests.
Reverse proxy	Server that receives client requests and forwards them to backends.
SAN	Subject Alternative Name; certificate field listing valid hostnames.
SNI	Server Name Indication; hostname sent in ClientHello.
TLS termination	Decrypting TLS at an intermediate node (the edge).
TTL	Time to live; how long a DNS answer may be cached.
VRRP	Protocol for a floating virtual IP shared by redundant routers/servers.

Appendix E

Execution Plan for Completion

Order	Time	Activity
1	15 min	Task A: connect Macs, collect addresses, ping matrix, topology
2	20 min	Task C: start both backends, verify from another Mac
3	20 min	Task B: dnsmasq on Mac 1, configure two clients, verify with dig
4	40 min	Tasks D+E: certificate/CA, nginx config, trust CA on clients, test without -k
5	10 min	Task F: Cache-Control, ETag, 304, browser cache hit
6	20 min	Task G: Wireshark captures (including TLS 1.2), save <code>.pcapng</code>
7	20 min	Five failure demonstrations with screenshots and explanations
8	60+ min	Phase 2: A, B, D, C (with rollback), E; troubleshooting practice
9	30 min	Fill evidence screenshots, finalise this document, repo, viva rehearsal

Table E.1: Suggested working order and time budget.